MEDIAFLOW

ADAPTIVE MULTIMEDIA STREAMING AND COMPRESSION SYSTEM

MULTIMEDIA TECHNOLOGY

A MINI PROJECT REPORT

Submitted by

THANVIR D - (73152522506)

SUHAIL THANVEER S - (73152522505)

VILSAN THEJA T - (73152522062)

in partial fulfilment of the requirements for

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

K.S.R. COLLEGE OF ENGINEERING

(AUTONOMOUS)

TIRUCHENGODE - 637 215

NOVEMBER 2026

K.S.R. COLLEGE OF ENGINEERING (AUTONOMOUS)

TIRUCHENGODE – 637 215

ANNA UNIVERSITY :: CHENNAI – 600 025

BONAFIDE CERTIFICATE

Certified that this mini project report entitled "ADAPTIVE MULTIMEDIA STREAMING AND COMPRESSION SYSTEM" is the bonafide work of the following students, who carried out the project work for Multimedia Technology under the guidance and supervision of the faculty members of the Department of Artificial Intelligence and Data Science. GOKUL PRASATH G (73152522016), NAVEEN K (73152522504), and YOGESHWARAN V (73152522507).

SIGNATURE
Mr. V. Karthi, M.E.
SUPERVISOR
Assistant Professor
Department of Artificial Intelligence and Data Science

SIGNATURE
Dr. Malatthi Sivasundaram, M.E., Ph.D.
HEAD OF THE DEPARTMENT
Associate Professor and Head of the Department
Department of Artificial Intelligence and Data Science

Submitted for the project viva-voce examination held on _____.

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to the management of K.S.R. College of Engineering for providing the academic environment, facilities, and encouragement necessary to undertake this mini project. The opportunity to explore multimedia technology through practical software development has been a valuable learning experience.

We convey our heartfelt thanks to our respected Principal and the faculty members of the Department of Artificial Intelligence and Data Science for their continued support and for fostering an environment that encourages innovation, collaboration, and problem-solving.

We are especially grateful to our project guide, Mr. V. Karthi, M.E., Assistant Professor, Department of Artificial Intelligence and Data Science, for his guidance, constructive suggestions, and encouragement throughout the development of the ADAPTIVE MULTIMEDIA STREAMING AND COMPRESSION SYSTEM. His guidance helped us refine our ideas and approach the project in a systematic manner.

We extend our sincere thanks to Dr. Malatthi Sivasundaram, M.E., Ph.D., Associate Professor and Head of the Department of Artificial Intelligence and Data Science, for her support and encouragement. We also acknowledge the faculty members, non-teaching staff, classmates, and everyone who contributed directly or indirectly to the progress of this work.

Finally, we thank our families for their patience, confidence, and continuous encouragement, which supported us throughout the completion of this project.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND DATA SCIENCE)

K.S.R. COLLEGE OF ENGINEERING (AUTONOMOUS)

TIRUCHENGODE - 637 215

NOVEMBER 2026

DECLARATION

We hereby declare that the mini project report entitled "ADAPTIVE MULTIMEDIA STREAMING AND COMPRESSION SYSTEM," submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Artificial Intelligence and Data Science, is a record of the work carried out by us as part of the Multimedia Technology project.

We further declare that this report has been prepared for academic purposes and that any external materials, references, or resources used in its preparation will be acknowledged appropriately. The report has not been submitted, in whole or in part, for the award of any other degree or qualification.

Place: Tiruchengode

Date: _____

THANVIR D (73152522506)

SUHAIL THANVEER S (73152522505)

VILSAN THEJA T (73152522062)

ABSTRACT

Digital images, audio and video now form most of the content that people create, share and study, yet raw multimedia files are large, formats are inconsistent and playback quality depends heavily on the viewer's device and network. Students and small content teams often lack a simple way to prepare media so that it plays smoothly on both fast and slow connections. This project proposes MediaFlow, an adaptive multimedia streaming and compression system that brings together media upload, automated transcoding into multiple quality levels, adaptive playback and measurable compression reporting. The initial prototype is designed as a responsive web application using React.js for the user interface, Python Flask for REST API services, FFmpeg for media processing and MySQL for structured data storage. Users can upload a video, audio or image file, after which the system validates it, extracts metadata, creates a thumbnail and generates several renditions at different resolutions and bitrates. The renditions are segmented into HTTP Live Streaming (HLS) playlists so that the player can switch quality according to available bandwidth. Captions in WebVTT format, quality selection and keyboard-accessible controls are included in the player design. For each processed file, the system reports original size, output size, compression ratio and, where a reference is available, objective quality measures such as PSNR and SSIM. These figures are presented as measurements of a specific encoding configuration and not as guarantees for other content. Access control, upload validation, storage limits and content-ownership checks are part of the proposed design. The report describes the problem, related approaches, architecture, workflow, implementation plan, sample scenarios, validation cases and future enhancements. The proposed system aims to make multimedia preparation, delivery and quality evaluation easier to understand and use in one accessible interface.

Keywords: Multimedia technology, video compression, adaptive bitrate streaming, HLS, FFmpeg, PSNR, SSIM, accessibility.

TABLE OF CONTENTS

ABSTRACT **5**
LIST OF ABBREVIATIONS **7**
CHAPTER 1 - INTRODUCTION **8**
1.1 Introduction of the Project 8
1.2 Need of the Project 8
1.3 Objectives of the Project 10
1.4 Theme of the Project 10
1.5 Organization of the Report 11
1.6 Summary 11
CHAPTER 2 - LITERATURE REVIEW **12**
2.1 Introduction 12
2.2 Existing Systems and Limitations 12
2.3 Scope of the Project 14
2.4 Summary 14
CHAPTER 3 - SYSTEM DESIGN AND DEVELOPMENT **16**
3.1 Multimedia Fundamentals 16
3.2 Proposed System Architecture 17
3.3 System Workflow 18
3.4 Implementation Details 20
3.5 Summary 22
CHAPTER 4 - RESULT AND DISCUSSION **23**
4.1 Functional Analysis 23
4.2 Test Plan and Expected Outputs 24
4.3 Illustrative User Scenario 25
4.4 Limitations and Risk Controls 25
4.5 Summary 26
CHAPTER 5 - CONCLUSION AND FUTURE SCOPE **27**
5.1 Conclusion 27
5.2 Future Scope 27
5.3 Applications 28
5.4 Summary 29
REFERENCES **30**

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AAC	Advanced Audio Coding
ABR	Adaptive Bitrate
AI	Artificial Intelligence
API	Application Programming Interface
CDN	Content Delivery Network
CRF	Constant Rate Factor
DASH	Dynamic Adaptive Streaming over HTTP
DBMS	Database Management System
FPS	Frames Per Second
HLS	HTTP Live Streaming
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JPEG	Joint Photographic Experts Group
MP4	MPEG-4 Part 14 container format
MPEG	Moving Picture Experts Group
PSNR	Peak Signal-to-Noise Ratio
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
SSIM	Structural Similarity Index Measure
UI	User Interface
UX	User Experience
WCAG	Web Content Accessibility Guidelines
WebVTT	Web Video Text Tracks

CHAPTER 1 - INTRODUCTION

1.1 INTRODUCTION OF THE PROJECT

Multimedia technology combines text, images, audio, video and animation into a single digital experience. Lectures are recorded as videos, notes are shared as audio, and learning material is increasingly designed to be seen and heard rather than only read. For this content to reach its audience reliably, it must be captured, compressed, stored, delivered and played back in a form that suits the viewer's device and network. A file that plays smoothly on a campus network may stall on a mobile connection, and a recording that is perfectly sharp may be too large to share.

The project MediaFlow proposes a web-based system that combines media upload, automated transcoding, adaptive streaming and compression reporting. Rather than asking the user to understand codecs and command-line tools, the system accepts a media file, checks it, prepares several quality levels and presents a player that chooses a suitable level automatically while still allowing manual selection. After processing, the user can see how much the file size was reduced and which encoding settings were used.

A second focus of the project is making multimedia content accessible and measurable. The player supports captions and keyboard control, and the system records objective quality measures so that a user can judge the trade-off between file size and visual quality. These measures describe one specific encoding result; the system does not claim that the same settings will suit all kinds of content.

The proposed prototype uses React.js for the responsive frontend, Python Flask for backend services, FFmpeg for media processing and MySQL for structured records. The design separates the upload interface, processing pipeline, streaming output, quality analysis and access control so that each module can be tested and improved independently.

1.2 NEED OF THE PROJECT

The need for MediaFlow arises from the gap between creating a media file and delivering it in a form that people can actually use. Students and small teams may record a lecture or demonstration but may not know which format, resolution or bitrate is appropriate,

how to add captions, or how to check whether a compressed copy still looks acceptable. A central and simple interface can reduce these technical barriers.

1.2.1 Large File Sizes and Storage Cost

Uncompressed or lightly compressed media is very large. Even a short recording can run into hundreds of megabytes, which makes uploading, sharing and storing it slow and costly. Compression reduces size by removing redundancy and, in lossy codecs, information that is less noticeable to human perception. The system should apply sensible defaults and report what was achieved.

1.2.2 Variable Network and Device Conditions

Viewers use phones, laptops and shared computers on networks of very different speed. A single high-bitrate file buffers on slow connections, while a single low-bitrate file wastes quality on fast ones. Adaptive bitrate streaming offers several renditions of the same content and lets the player switch between them as conditions change.

1.2.3 Format Incompatibility

Media is produced in many containers and codecs, and not every browser plays every combination. Converting uploads into widely supported formats such as H.264 video with AAC audio inside standard containers reduces playback failures and avoids asking the viewer to install additional software.

1.2.4 Accessibility and Inclusion

Video without captions excludes viewers who are deaf or hard of hearing, and also those who watch in noisy places or without sound. Controls that need a mouse exclude keyboard-only users. Accessibility therefore has to be part of the player design rather than an afterthought.

1.2.5 Need for Measurable Quality

Judging compression quality by eye alone is subjective. Objective measures such as PSNR and SSIM give a repeatable comparison between an original and its compressed version. They do not replace human judgement, but they help a user decide whether an encoding setting is acceptable and document the decision.

1.3 OBJECTIVES OF THE PROJECT

- To design a user-friendly system for uploading and managing image, audio and video content.
- To validate uploaded files for type, size and integrity before processing.
- To transcode video into multiple renditions and package them as HLS playlists for adaptive playback.
- To provide a web player with quality selection, captions and keyboard-accessible controls.
- To report original size, output size and compression ratio for every processed file.
- To compute objective quality measures such as PSNR and SSIM where a reference file is available.
- To provide basic safeguards including authentication, ownership checks and storage limits.
- To evaluate usability, input validation, processing, playback and access controls through a documented test plan.
- To identify future extensions such as DASH output, hardware acceleration and live streaming.

1.4 THEME OF THE PROJECT

The theme of MediaFlow is accessible, efficient and measurable multimedia delivery. The project combines standard compression and streaming techniques with a simple web interface so that users can prepare media, watch it smoothly across different conditions and understand the effect of compression on size and quality.

1.4.1 User-Centred Design

The workflow is designed around a user's immediate question: "Will this play well for the people I want to share it with?" The upload form should be short, processing progress should be visible, and the result should be a link and a clear summary rather than a list of encoder options. Advanced settings can be available but hidden by default.

1.4.2 Use of Established Standards

In the initial prototype, the system relies on widely supported standards: H.264 for video, AAC for audio, JPEG and WebP for images, HLS for streaming and WebVTT for captions. Newer codecs can be added later when browser support and processing cost justify them. Choices are explained to the user in plain language.

1.4.3 Accessibility and Inclusion

The interface should work on mobile phones and computers, use readable text, provide captions or text alternatives for audio and video content where possible, and avoid relying on colour alone to communicate status. English may be used for the first prototype, with Tamil and other language support considered as a future enhancement.

1.5 ORGANIZATION OF THE REPORT

This report is organized into five chapters. Chapter 1 introduces the problem, need, objectives and theme of the project. Chapter 2 reviews related multimedia platforms and their limitations. Chapter 3 explains the multimedia fundamentals, proposed system architecture, workflow, modules, data design and implementation plan. Chapter 4 describes functional expectations, planned test cases, an illustrative user scenario and risk controls. Chapter 5 presents the conclusion, future scope and possible applications. References are listed at the end.

1.6 SUMMARY

This chapter introduced MediaFlow as a proposed adaptive multimedia streaming and compression system. The main motivation is to connect media upload, automated transcoding, adaptive playback and measurable quality reporting in a single user-friendly workflow. Accessibility, transparency about compression results and the use of open standards are essential design principles. These requirements guide the review of existing systems and the proposed design in the following chapters.

CHAPTER 2 - LITERATURE REVIEW

2.1 INTRODUCTION

A literature review helps identify established practices, common features and remaining gaps in multimedia processing and delivery. Relevant approaches include image and video compression standards, adaptive streaming protocols, online video platforms, media-conversion tools and accessibility guidelines. These address different needs; some focus on reducing size, some on reliable delivery and others on making content usable by everyone.

Still-image compression was standardized by the JPEG committee, whose baseline method uses the discrete cosine transform, quantization and entropy coding [1]. Video compression extends these ideas with motion estimation between frames. The H.264/AVC standard became the most widely deployed video codec and is supported by nearly all browsers and devices [2], while High Efficiency Video Coding (HEVC) was designed to deliver similar quality at a lower bitrate at the cost of higher computational effort [3]. For delivery, HTTP Live Streaming describes how media is divided into short segments listed in playlists so that a player can request them over ordinary web servers [4], and MPEG-DASH defines a comparable open standard [5]. The Web Content Accessibility Guidelines set out requirements such as captions and keyboard operability for media content [6]. These sources support the project's decision to use standard codecs, segmented adaptive streaming and an accessible player.

2.2 EXISTING SYSTEMS AND THEIR LIMITATIONS

2.2.1 Large Video-Sharing Platforms

Public video platforms handle transcoding and adaptive streaming automatically and at very large scale. They are convenient for sharing, but the user has little control over encoding settings, cannot see the compression results and must accept the platform's hosting, privacy and advertising terms. MediaFlow aims to expose a simplified view of what happens during processing and to keep content under the user's control.

2.2.2 Command-Line Media Tools

FFmpeg and similar tools are powerful and can perform nearly every conversion needed for this project [7]. Their limitation for the target users is the learning curve: choosing codecs, rate-control modes and segment settings requires technical knowledge. MediaFlow proposes using FFmpeg as the processing engine while presenting its capabilities through a guided web interface.

2.2.3 Desktop Converters and Editors

Desktop converters allow users to change format and size but usually produce a single output file. They do not create adaptive renditions, do not publish a streamable link and rarely report objective quality. A user who needs several versions must repeat the process manually.

2.2.4 Learning Management System Video Players

Learning platforms embed video but often leave preparation to the content creator. Large uploads may be rejected, captions may be optional or missing, and playback on weak connections can be poor. MediaFlow proposes preparing media before it is placed in such systems, with captions and accessible controls built into the player.

2.2.5 Objective Quality Metrics

PSNR measures pixel-level error between an original and a processed frame and is simple to compute, but it does not always match human perception. SSIM compares structural information and often correlates better with perceived quality. Both need a reference, and neither is a complete substitute for viewing the result. The system will therefore display these values together with the settings used and encourage the user to preview the output.

2.2.6 Key Gaps Identified

- Upload, compression, adaptive streaming and quality reporting are often provided by separate tools.
- Encoding settings and their effect on size and quality are rarely explained to non-expert users.
- Captions and keyboard-accessible controls may be missing from simple media pages.
- Large uploads may fail without clear validation messages or progress information.

- Privacy and ownership of uploaded content may be unclear on public platforms.
- Compression results can be misunderstood as guarantees if limitations are not stated.

2.3 SCOPE OF THE PROJECT

The scope of the initial MediaFlow prototype is to demonstrate a complete workflow from media upload to adaptive playback and compression reporting. The system focuses on accessibility, transparency of results and use of open standards rather than on large-scale commercial delivery.

2.3.1 Included in the Proposed Prototype

- User registration/sign-in or a controlled demonstration mode.
- Upload of video, audio and image files with type, size and integrity validation.
- Metadata extraction (duration, resolution, codec, bitrate) and thumbnail generation.
- Transcoding of video into several renditions using H.264 and AAC, with illustrative settings labelled as design choices.
- HLS packaging and a web player with automatic and manual quality selection.
- WebVTT caption upload and display.
- Compression summary showing sizes, ratio and, where feasible, PSNR and SSIM.
- A personal media library showing only the signed-in user's own files.
- Input validation, secure authenticated endpoints and access restrictions.

2.3.2 Outside the Initial Scope

- Live streaming and real-time conferencing.
- Digital rights management and paid content delivery.
- Automatic speech recognition for caption generation, unless added later as a separate reviewed module.
- Large-scale content delivery network deployment.
- Guarantees that a particular setting will give acceptable quality for every kind of content.
- Hosting or distribution of copyrighted material without the owner's permission.

2.4 SUMMARY

This chapter reviewed the standards and categories of tools that support multimedia compression and delivery. The review identified an opportunity to combine upload validation, automated transcoding, adaptive streaming, accessible playback and measurable quality reporting. It also highlighted the importance of using established standards, stating the limits of quality metrics and protecting users' content. These points inform the architecture and implementation plan presented next.

CHAPTER 3 - SYSTEM DESIGN AND DEVELOPMENT

3.1 FUNDAMENTALS OF MULTIMEDIA PROCESSING AND DELIVERY

A multimedia delivery system captures or receives media, converts it into efficient digital representations, stores it and delivers it to a player. The design must balance quality, file size, processing time and compatibility. The proposed MediaFlow prototype relies on existing standards and tools, and its contribution lies in combining them in a guided and measurable workflow.

3.1.1 Digital Media Types

Images are grids of pixels described by resolution and colour depth. Audio is a sequence of samples characterized by sampling rate and bit depth. Video is a sequence of images shown at a frame rate, usually accompanied by audio. Each type can be stored in different containers and encoded with different codecs, and the choice affects size, quality and compatibility.

3.1.2 Compression

Lossless compression allows exact reconstruction, while lossy compression discards information that is less perceptible in order to achieve a much smaller size. Video codecs exploit spatial redundancy within a frame and temporal redundancy between frames. The constant rate factor (CRF) mode targets a consistent perceptual quality, whereas a fixed bitrate mode targets a predictable data rate, which is useful for streaming ladders. The system uses settings chosen for each rendition and records them with the result.

3.1.3 Adaptive Bitrate Streaming

In adaptive bitrate streaming, the source is encoded at several quality levels, and each level is divided into segments of a few seconds. A playlist lists the available levels and their segments. The player measures download speed and buffer level and requests segments from the most suitable level, switching up or down as conditions change. This reduces stalling without requiring the user to choose a quality in advance.

3.1.4 Accessible Playback

Accessible playback includes synchronized captions, text alternatives for non-text content, keyboard operation of every control, visible focus indicators and sufficient colour contrast. Captions are stored as WebVTT files linked to the media record and loaded by the player as text tracks.

3.2 PROPOSED SYSTEM ARCHITECTURE

MediaFlow follows a modular web architecture. The frontend collects uploads and displays results. The Flask backend validates requests, creates processing jobs and controls access to stored data. The media-processing layer runs FFmpeg to generate renditions, segments, thumbnails and quality measures. A storage layer holds original files and processed outputs, and MySQL stores the records needed for the application. An external content delivery or storage service may be integrated only when its terms, data protection requirements and reliability have been reviewed.

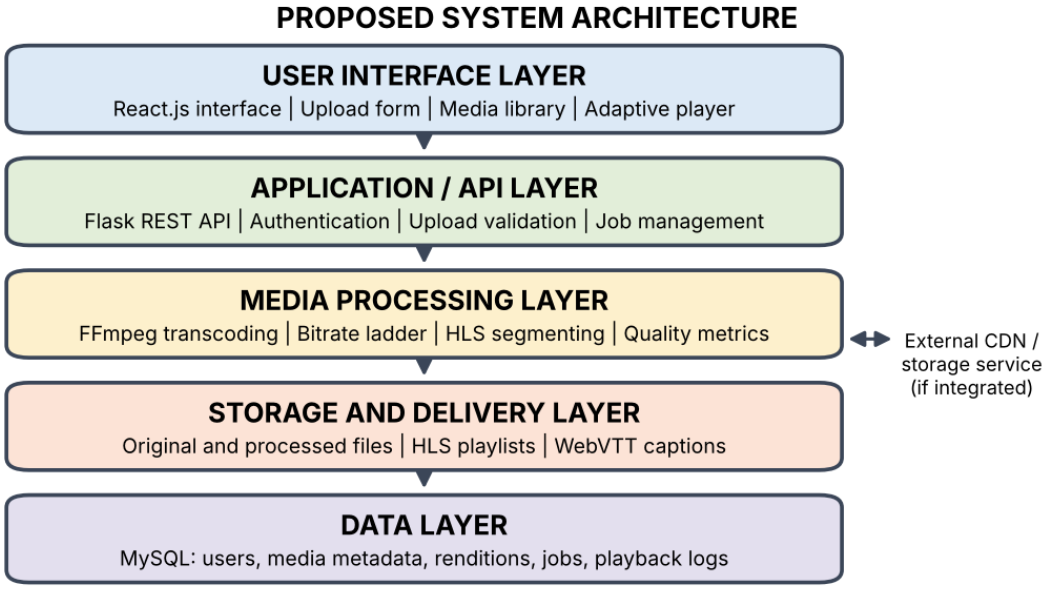


Figure 3.1: Proposed layered architecture of MediaFlow

3.2.1 User Interface Layer

React.js is proposed for a responsive interface that can be used on mobile phones, tablets and computers. Main screens include sign-in, upload, processing status, media library, player page and compression report. Readable labels, keyboard navigation, captions and non-

colour indicators should be included wherever possible. An HLS-capable player library is used where the browser does not play HLS natively.

3.2.2 Backend API Layer

Python Flask exposes REST endpoints for registration, upload, job status, media listing, playback information, caption upload, compression report and deletion. The backend validates data before storage, enforces authentication and returns predictable error messages. Secrets such as database passwords and API keys should be kept in environment variables rather than committed to source control.

3.2.3 Media Processing Layer

Processing runs outside the request cycle, in a background worker, so that long encodes do not block the interface. The worker calls FFmpeg with fixed, reviewed command templates; user-supplied text is never inserted directly into a command. For each video it extracts metadata, creates a thumbnail, produces the configured renditions, segments them into HLS and optionally computes PSNR and SSIM against the source. Failed jobs are marked with a clear reason and can be retried.

3.2.4 Storage and Data Layer

Original files are kept in a protected storage location, and processed outputs are placed in a separate location served through controlled URLs. MySQL stores account metadata, media records, renditions, processing jobs, captions and playback logs. Database access must be limited to authenticated application services. Files and records should be retained only as long as needed and deleted when the owner requests it.

3.3 SYSTEM WORKFLOW

The proposed workflow is shown in Figure 3.2. The user begins by signing in and selecting a file. The system validates it, extracts metadata, transcodes it into several renditions and packages the output for streaming. The user can then play the media, compare compression results and manage the library.

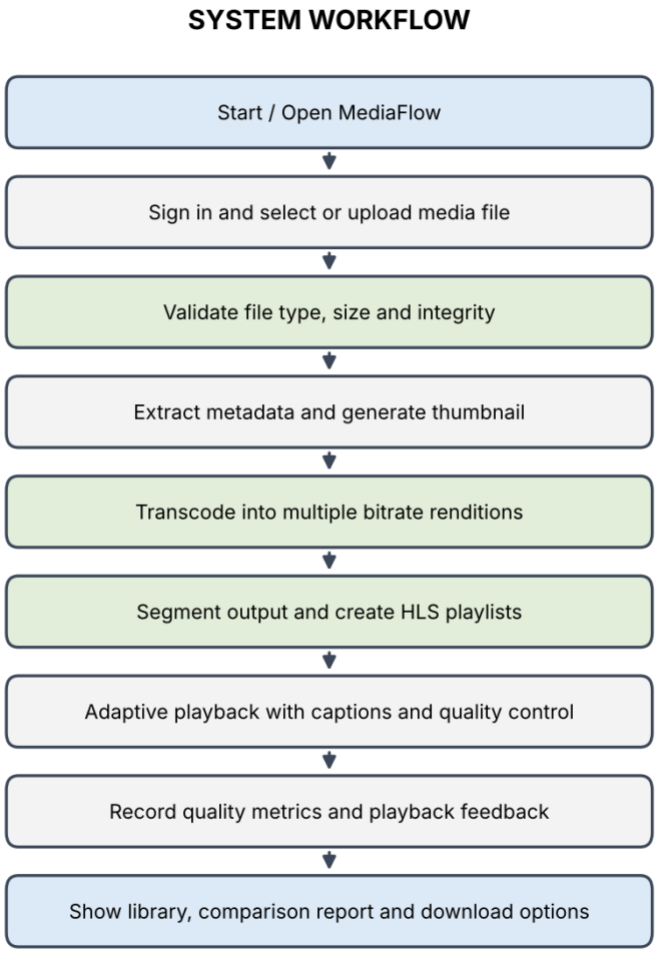


Figure 3.2: Proposed user workflow

3.3.1 Step-by-Step Process

1. Open the application and sign in or continue in an approved demonstration mode.
2. Select a video, audio or image file and optionally add a caption file and title.
3. The backend checks file type, size limit and integrity, and stores the original under the user's account.
4. A processing job is created and the interface shows its status.
5. The worker extracts metadata and generates a thumbnail.
6. The worker transcodes video into the configured renditions and segments them into HLS playlists.
7. Optional quality measures (PSNR and SSIM) are calculated and stored with the job.
8. The user opens the player, which starts at a suitable quality and can switch automatically or manually.

9. The user reviews the compression report and can download, share within permitted limits or delete the media.

3.4 IMPLEMENTATION DETAILS

3.4.1 Technology Stack

COMPONENT	PROPOSED TECHNOLOGY	RESPONSIBILITY
Frontend	React.js, HTML, CSS, JavaScript	Responsive pages, upload form, library and player
Backend	Python Flask REST API	Request processing, validation, job management and authorization
Media processing	FFmpeg (command templates run by a worker)	Metadata, thumbnails, transcoding, segmenting and quality measures
Streaming format	HLS with H.264 video and AAC audio	Adaptive playback in common browsers
Captions	WebVTT text tracks	Synchronized, accessible captions
Database	MySQL	Structured storage for accounts, media, renditions and jobs
Testing	Pytest / API checks / browser testing	Validate functional behaviour, error handling and access controls

3.4.2 Main Functional Modules

MODULE	DESCRIPTION	EXPECTED OUTPUT
Account and Access	Handles sign-in, sessions and ownership of media.	Authorized session and per-user library.
Upload and Validation	Checks type, size and integrity of uploaded files.	Accepted file or clear error message.
Transcoding Engine	Creates renditions and HLS segments using FFmpeg.	Playlists and segment files for each quality level.
Streaming Player	Plays HLS with quality selection and captions.	Smooth playback with accessible controls.
Quality Analysis	Calculates size ratio, PSNR and SSIM where possible.	Compression report for the selected settings.
Media Library	Lists, searches and deletes a user's media.	Personal library view.
Safety and Content Controls	Limits storage, blocks unsupported files and honours deletion.	Controlled use of storage and clear ownership.

3.4.3 Example Encoding Ladder

The following ladder is an illustrative design choice for the first prototype. The values are not universal recommendations, and the best settings depend on the content and audience. The system never lets a rendition exceed the resolution of the source.

RENDITION	RESOLUTION	VIDEO BITRATE (ILLUSTRATIVE)	AUDIO	NOTE
Low	426 x 240	about 400 kbps	AAC 64 kbps	Weak mobile connections.
Medium	640 x 360	about 800 kbps	AAC 96 kbps	Default for small screens.
Standard	854 x 480	about 1400 kbps	AAC 128 kbps	General viewing.
HD	1280 x 720	about 2800 kbps	AAC 128 kbps	Laptops and larger screens.
Full HD	1920 x 1080	about 5000 kbps	AAC 128 kbps	Only if the source is at least 1080p.

3.4.4 Proposed Database Design

TABLE	IMPORTANT FIELDS	PURPOSE
Users	user_id, display_name, email/login identifier, created_at	Account identity and basic account metadata.
Media_Files	media_id, user_id, title, media_type, original_path, duration, width, height, size_bytes, status, created_at	Stores uploaded media and its metadata.
Renditions	rendition_id, media_id, label, resolution, bitrate, codec, playlist_path, size_bytes	Stores each processed quality level.
Processing_Jobs	job_id, media_id, state, started_at, finished_at, error_message	Tracks transcoding progress and failures.
Quality_Reports	report_id, media_id, rendition_id, compression_ratio, psnr, ssim, settings_summary	Stores objective measurements for a specific encode.
Captions	caption_id, media_id, language, file_path, created_at	Links WebVTT files to media.
Playback_Logs	log_id, media_id, session_id, rendition_label, stall_count, created_at	Optional, minimal data for evaluating playback.

The field list is a proposed schema and can be adjusted after implementation. Passwords must never be stored as plain text. Access tokens or session identifiers should be protected, and user queries should use parameterized database operations. Records should be retained and deleted according to a clear policy.

3.4.5 Processing Pseudocode

INPUT: uploaded file, user account, encoding ladder

1. Validate file type, size limit and integrity; verify the user owns the upload.
2. Store the original file in protected storage and create a media record.
3. Extract metadata (duration, resolution, codec, bitrate) and create a thumbnail.
4. Remove ladder levels that are larger than the source resolution.
5. For each remaining level, transcode with the reviewed FFmpeg template.
6. Segment each level and write HLS playlists and a master playlist.
7. If enabled, compute PSNR and SSIM against the source and store the results.
8. Mark the job complete or failed and update the media status.
9. Serve the master playlist to the player through an authorized URL.

NOTE: Reported quality values describe the tested encode only and do not predict other content.

3.4.6 Security, Privacy and Content Controls

- Accept only listed file types, check content signatures and enforce size and duration limits.

- Run FFmpeg from fixed command templates with arguments passed as separate values, never through shell string concatenation.
- Use HTTPS in deployment, secure session handling, password hashing and role-based access control.
- Store originals outside the public web directory and serve outputs through authorized URLs.
- Allow users to review and delete their media, renditions and captions.
- Collect only the playback data needed for evaluation and avoid storing personal details in logs.
- Apply per-user storage quotas and process limits to prevent misuse of resources.
- Require users to confirm that they have the right to upload and share the content.

3.5 SUMMARY

This chapter described MediaFlow's multimedia fundamentals, proposed architecture and implementation plan. The system uses a responsive frontend, Flask APIs, an FFmpeg-based processing layer, HLS packaging, an accessible player and MySQL storage. The workflow includes upload validation, transcoding, adaptive playback and compression reporting. The design emphasizes standards, security and clear limits on what the quality measurements can show.

CHAPTER 4 - RESULT AND DISCUSSION

4.1 FUNCTIONAL ANALYSIS

The functional analysis identifies the behaviours that the prototype should demonstrate. Since implementation logs and measured test results were not available when this report was prepared, this report defines expected behaviour and a validation plan rather than claiming that all tests have already passed. Actual status, screenshots and measured results should be inserted after the application has been implemented and tested.

4.1.1 Upload Handling

The application should accept supported files within the configured limits, reject unsupported, empty, oversized or corrupted files with a clear message, and store the upload only for an authenticated user. The confirmation screen should show the file name and processing status without revealing any other user's information.

4.1.2 Transcoding and Packaging

For a valid video, the system should create the configured renditions, skipping any that exceed the source resolution, and produce a master playlist that references each of them. A failed job should be reported with a reason and should not leave a media record marked as ready. The original file should remain unchanged.

4.1.3 Playback and Accessibility

When a user opens a processed item, the player should start, offer automatic and manual quality selection and allow pause, seek and volume control from both mouse and keyboard. Captions should appear when a caption file is attached. Playback behaviour on slow connections should be observed during testing rather than assumed.

4.1.4 Compression Reporting

The report should show the original size, the size of each rendition, the resulting ratio and, when computed, PSNR and SSIM with the settings used. It should not show a guaranteed improvement, and it should state that the values apply to the tested file and configuration.

4.1.5 Library and Privacy

The library should show only the signed-in user's own media. Another user's media must not be accessible by guessing a URL. The user should be able to delete an item, and the associated files and records should be removed or the documented retention process followed.

4.2 TEST PLAN AND EXPECTED OUTPUTS

The following table provides the initial functional test plan. The status is intentionally written as "To be tested" because no executed test logs were available. Once the prototype is running, the team should record the actual status, date, test environment and evidence for each case.

TEST ID	TEST CATEGORY	TEST INPUT / ACTION	EXPECTED OUTPUT	STATUS
T01	Upload validation	Upload a supported video within limits.	File is accepted and a processing job is created.	To be tested
T02	Type validation	Upload an unsupported or renamed file.	File is rejected with a clear message; nothing is stored.	To be tested
T03	Size limit	Upload a file above the size limit.	Upload is rejected with the permitted limit shown.	To be tested
T04	Transcoding	Process a 1080p sample video.	Renditions are created and listed in the master playlist.	To be tested
T05	Resolution rule	Process a 480p sample video.	No rendition above 480p is generated.	To be tested
T06	Adaptive playback	Play a processed video and throttle the network.	Player switches to a lower rendition without stopping when possible.	To be tested
T07	Manual quality	Select a specific quality in the player.	Selected rendition plays until the user changes it.	To be tested
T08	Captions	Attach a WebVTT file and play.	Captions appear in sync and can be turned off.	To be tested
T09	Keyboard access	Operate the player using the keyboard only.	All controls are reachable and focus is visible.	To be tested
T10	Compression report	Open the report of a processed file.	Sizes and ratio are correct; PSNR/SSIM appear if computed.	To be tested
T11	Library privacy	Attempt to open another user's media.	Access is denied.	To be tested
T12	Failed job	Process a corrupted video.	Job is marked failed with a reason; no ready item is created.	To be tested
T13	Responsive layout	Open screens on mobile and desktop widths.	Content remains readable and controls usable.	To be tested
T14	Deletion	Delete a media item.	Files and records are removed or the retention process is followed.	To be tested

4.3 ILLUSTRATIVE USER SCENARIO

The following example demonstrates how the proposed workflow may operate. It is an illustrative scenario, not a measured experiment.

Scenario: Student Sharing a Recorded Lab Demonstration

STEP	USER ACTION	SYSTEM RESPONSE
1	The user signs in and selects a 1080p recorded demonstration video.	The application checks the type, size and integrity of the file and creates a processing job.
2	The user adds a title and a WebVTT caption file.	The metadata is stored and the caption is linked to the media record.
3	The user waits while processing runs.	The status changes from queued to processing to ready; a thumbnail appears in the library.
4	The user opens the player on a laptop.	Playback begins at a suitable quality and the player offers automatic and manual quality selection.
5	A classmate opens the same link on a mobile connection.	The player starts at a lower rendition and may move up if conditions improve.
6	The user opens the compression report.	The report lists the original size, rendition sizes and ratio, with the encoding settings used.

Example interface message: “Your video is ready. It was prepared in four quality levels. You can preview it, add captions or view the compression report.” The message states what happened without promising a particular viewing experience.

4.4 LIMITATIONS AND RISK CONTROLS

4.4.1 Limitations

- Transcoding is computationally heavy; processing time depends on file length and server capacity.
- Illustrative bitrate values may not suit all content; fast-motion or detailed footage may need higher rates.
- PSNR and SSIM do not capture every aspect of perceived quality.
- Browser support for streaming and captions may vary across devices.
- Playback quality depends on the user’s network and device.
- Without real implementation and user testing, performance and usability claims remain unmeasured.

4.4.2 Risk Controls

- Validate every upload and run processing in a background worker with time and resource limits.

- Use fixed FFmpeg command templates to prevent command injection.
- Protect media with authentication, authorization, secure transport and per-user quotas.
- Preserve originals and report failures clearly rather than showing partial results as complete.
- Display quality measurements together with their settings and limitations.
- Provide captions and keyboard operation, and test on both mobile and desktop screens.
- Review any future automated or machine-learning features before they are added.

4.5 SUMMARY

This chapter presented a functional validation plan, test cases and an illustrative user journey. The defined checks cover upload handling, transcoding, adaptive playback, captions, compression reporting, privacy and deletion. Test outcomes are pending execution; the report intentionally avoids fabricated measurements or claims of completed testing. Limitations and risk controls are also described to keep the proposed system safe and transparent.

CHAPTER 5 - CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

MediaFlow is proposed as an adaptive multimedia streaming and compression system that brings together media upload, automated transcoding, adaptive playback, accessible captions and measurable compression reporting. The project responds to a practical need: preparing media so that it is small enough to share and reliable enough to watch under different network conditions, while helping users understand what compression has done to their content.

The proposed architecture uses React.js, Python Flask, FFmpeg and MySQL. Established standards such as H.264, AAC, HLS and WebVTT are preferred for the initial prototype because they are widely supported and well documented. After processing, the user can play the media at an automatically selected quality, review the compression report and manage the personal library.

A central principle of the design is that compression results and quality measures describe a specific file and configuration and are not guarantees. Upload validation, protected storage, access control and accessible playback are important conditions for responsible use. Implementation and user testing are required before performance or usability claims can be made.

5.2 FUTURE SCOPE

5.2.1 Additional Codecs and Streaming Formats

Support for MPEG-DASH, HEVC and newer royalty-free codecs can be added where browser support and licensing permit. The system could then choose the most efficient supported format for each viewer.

5.2.2 Hardware-Accelerated Processing

Using GPU or dedicated encoder hardware can shorten processing time. Any such change should be tested to confirm that visual quality at the chosen settings is still acceptable.

5.2.3 Automatic Captioning

Automatic speech recognition can generate draft captions in English and Indian languages. Generated captions must be reviewed for accuracy before publication, and users should be told when captions are machine-generated.

5.2.4 Live Streaming

The pipeline can be extended to ingest live sources and produce low-latency adaptive streams. This requires additional work on scalability, reliability and monitoring.

5.2.5 Security and Delivery Improvements

Further improvements may include multi-factor authentication, signed expiring URLs, encrypted storage, virus scanning of uploads, content delivery network integration and automated retention controls. A formal security assessment should be completed before deployment with real users.

5.2.6 Accessibility and Evaluation

Future development can add audio descriptions, transcripts, screen-reader labels and low-bandwidth modes. Usability evaluation with volunteers can measure task completion, start-up time, stalls and satisfaction. Such studies should be conducted with appropriate ethical review.

5.3 APPLICATIONS

- College lecture-capture and e-learning portals that need smooth playback across devices.
- Student project showcases and laboratory demonstration repositories.
- Small organizations that need to publish training or promotional videos without a dedicated media team.
- Teaching tools for demonstrating compression, bitrate and quality trade-offs in multimedia courses.
- Community and public-information channels that need accessible, captioned video.
- Personal media libraries that need smaller files and consistent formats.

The application should not be used to distribute material without the owner's permission. If a deployment is used in a college or organization, uploaded media must be shared only according to the permissions set by its owner and the applicable policies.

5.4 SUMMARY

This chapter summarized MediaFlow and its future improvements. Its main contribution is to combine accessible adaptive streaming, automated compression and measurable quality reporting in a single, easy-to-use workflow.

REFERENCES

[1] G. K. Wallace, “The JPEG still picture compression standard,” IEEE Transactions on Consumer Electronics, vol. 38, no. 1, pp. xviii–xxxiv, Feb. 1992.

[2] T. Wiegand, G. J. Sullivan, G. Bjøntegaard and A. Luthra, “Overview of the H.264/AVC video coding standard,” IEEE Transactions on Circuits and Systems for Video Technology, vol. 13, no. 7, pp. 560–576, Jul. 2003.

[3] G. J. Sullivan, J.-R. Ohm, W.-J. Han and T. Wiegand, “Overview of the High Efficiency Video Coding (HEVC) standard,” IEEE Transactions on Circuits and Systems for Video Technology, vol. 22, no. 12, pp. 1649–1668, Dec. 2012.

[4] R. Pantos and W. May, “HTTP Live Streaming,” RFC 8216, Internet Engineering Task Force, Aug. 2017. Available: https://www.rfc-editor.org/rfc/rfc8216.

[5] I. Sodagar, “The MPEG-DASH standard for multimedia streaming over the Internet,” IEEE MultiMedia, vol. 18, no. 4, pp. 62–67, Apr. 2011.

[6] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.2.” Available: https://www.w3.org/TR/WCAG22/.

[7] FFmpeg Developers, “FFmpeg Documentation.” Available: https://ffmpeg.org/documentation.html.

[8] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600–612, Apr. 2004.

[9] React Team, “React Documentation.” Available: https://react.dev/learn.

[10] Pallets Projects, “Flask Documentation.” Available: https://flask.palletsprojects.com/.

[11] Oracle, “MySQL 8.4 Reference Manual.” Available: https://dev.mysql.com/doc/.